Kalman Filter

Contents

1. Overview . . . 2
2. Why two numbers are enough: the linear Gaussian system . . . 4
   - 2.1 The Bayes filter in brief . . . 4
   - 2.2 A Gaussian is two numbers . . . 4
   - 2.3 A linear function keeps a Gaussian a Gaussian . . . 5
   - 2.4 Adding independent errors adds variances . . . 8
   - 2.5 The linear Gaussian system . . . 9
3. Predict: the Gaussian moves and widens . . . 10
4. Update: the gain that makes the estimate most certain . . . 11
   - 4.1 Two estimates of one position . . . 11
   - 4.2 The variance of the blend . . . 12
   - 4.3 The best weight: the Kalman gain . . . 13
   - 4.4 The updated estimate and its variance . . . 14
   - 4.5 The same answer from multiplying Gaussians . . . 16
5. Ten steps on the corridor . . . 17
6. The matrix form: more than one number . . . 21
   - 6.1 Why a vector state: the hidden speed . . . 21
   - 6.2 The model as matrices . . . 21
   - 6.3 The five lines in matrix form . . . 22
   - 6.4 One step in matrix form, and the speed nobody measured . . . 23
   - 6.5 Observability: can the readings pin down the state? . . . 25
   - 6.6 The state a controller needs: the filter as an observer . . . 26
7. Choosing Q and R . . . 27
   - 7.1 R: measure the sensor . . . 27
   - 7.2 Q: tune the motion noise . . . 27
8. Summary . . . 30
9. Sources . . . 31
10. Key terms . . . 32

1. Overview

> **Key point:** A robot has two noisy ways to know where it is: its commands, which drift, and its sensor, which jumps. The Kalman filter blends them with the weight that makes the result as certain as possible, and it does so one step at a time from two numbers.

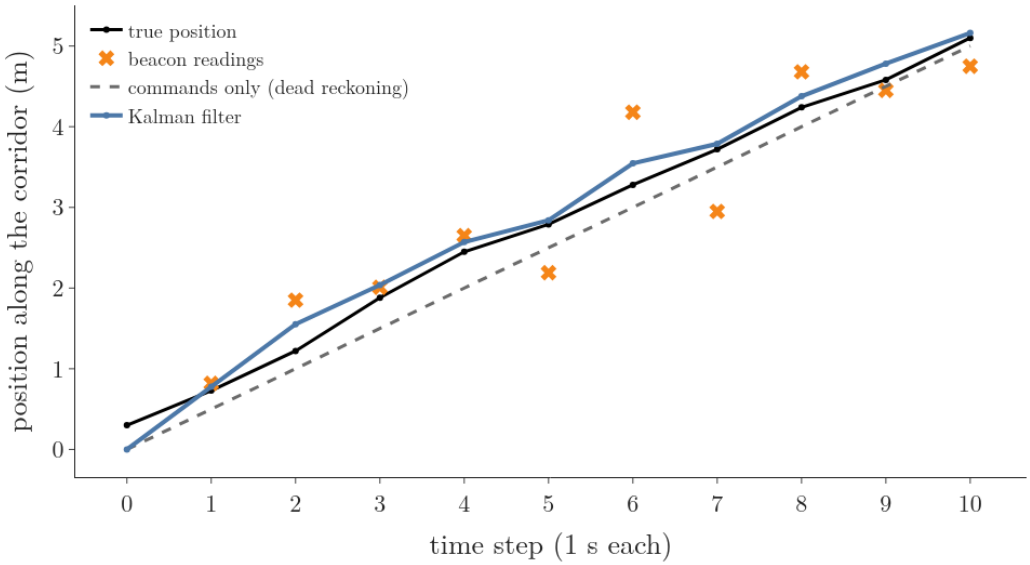


Figure 1: Ten steps of a robot in a corridor: the true position, the jumpy beacon readings, the drifting estimate from the commands alone, and the Kalman filter estimate that follows the truth

The **particle filter** (G-2492) lets the robot's **belief** (G-2433), its probability for each possible position, take any shape, but every particle must be moved and weighed every round. When the belief is a single bell-shaped curve, a **Gaussian** (G-827), two numbers carry all of it; this Note builds that filter, the **Kalman filter** (G-2506). It needs a new world. The three doors of the looped corridor give a belief with a peak at each door: three **modes** (G-1251), so the belief is **multimodal** (G-1275), and one Gaussian cannot hold it. Here a ceiling beacon reports the position itself, so the belief has a single mode: it is **unimodal** (G-2528).

A delivery robot drives along a straight corridor. Every second it is commanded to move 0.5 m. A ceiling beacon system also reports its position once a second. Neither source is good enough on its own (Figure 1):

- **The commands** give a smooth path, but they **drift**: each step's error is the last step's error plus a new wheel slip, so the error carries over and piles up. Over the ten steps of Figure 1, the commands alone miss the true position by 0.27 m on average (the **root mean squared error** (G-1705), RMSE).
- **The beacon jumps**: each reading's error is new, with no memory of the last one, so it never piles up, but it can swing from one side to the other between two seconds. Its readings miss by 0.51 m on average.

Figure 2 runs both sources for 60 steps from a known start. The commands' error moves only a little each step, but the moves add up, and its band of two **standard deviations** (G-1871), twice the error's typical size, widens with the number of steps (Section 2.4). With:

- t is the number of steps since the start, from 1 to 60 in Figure 2;
- 0.1 m is the standard deviation of one step's wheel slip.

The band reaches this far either side of zero, in m:

$$2 \times 0.1 \times \sqrt{t}$$

The beacon's error lands somewhere new each step, but its band stays at ±0.8 m. After about 16 steps the commands are the worse source.

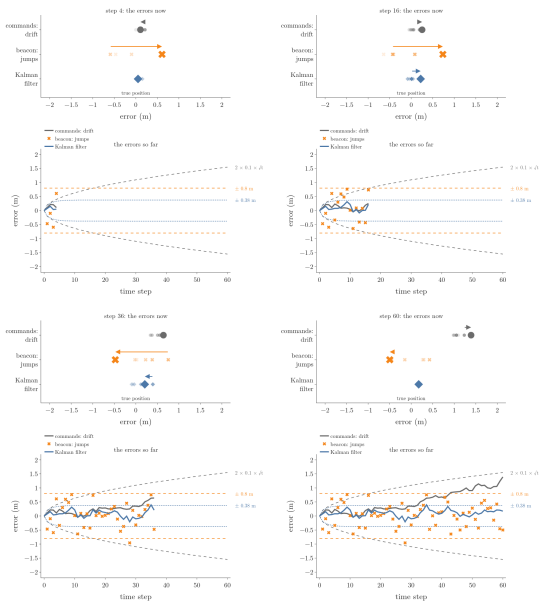


Figure 2: One simulated run of 60 steps from a known start, one frame per step. Top: the errors now, with an arrow from each last value; the commands' error moves a little each step and wanders off, the beacon's lands somewhere new each step, the Kalman filter's stays near zero. Bottom: the errors so far; the commands' band of two standard deviations widens, the beacon's stays at ±0.8 m, the Kalman filter's settles at ±0.38 m

Over the ten steps of Figure 1, the Kalman filter's estimate misses by 0.17 m, better than either (notebook). In Figure 2 (blue), its error stays near zero, and its band settles at ±0.38 m, half the beacon's, and does not widen. The filter gets there by asking, at every step, how far to trust each source, and the answer is a number we derive: the **Kalman gain** (G-2508). The Note builds the filter on this corridor, one step at a time, in the order of kalmanfilter.net and Becker's book:

- why a Gaussian belief stays Gaussian, so two numbers are enough (Section 2);
- the predict step, which carries the drift: the Gaussian moves and widens (Section 3);

- the update step, which uses the beacon: the gain derived as the weight that makes the estimate's **variance** (G-2076), the spread of its error, smallest (Section 4);
- the ten steps of Figure 1 by hand (Section 5);
- the matrix form, which estimates a speed that is never measured, then **observability** (G-2517), whether the readings can pin down the whole state, and the filter as the source of the state a controller needs (Section 6);
- how to choose the two noise settings: Q, the variance of the wheel slip, 0.01 m^2, and R, the variance of the beacon's error, 0.16 m^2 (Section 7).

2. Why two numbers are enough: the linear Gaussian system

> **Key point:** To weigh the drifting commands against the jumping beacon at every step, the filter must carry its **belief** (G-2433) in a few numbers. If the motion and the sensor are **linear** functions (G-2527) of the state (the output y is a fixed multiple a of the state x plus a constant b, $y = ax + b$, whose graph is a straight line) with added Gaussian noise, a Gaussian belief stays Gaussian, so a mean and a **variance** (G-2076) carry the whole belief.

2.1 The Bayes filter in brief

The robot's **belief** (G-2433) is a probability distribution over its position. The **Bayes filter** (G-2446) updates it once a second, in rounds numbered by the step t. At step t:

- x_t is the robot's position, in m;
- $\text{bel}(x_t)$ is the belief, the probability distribution over the position x_t;
- u_t is the command, 0.5 m forward;
- z_t is the beacon's reading, in m;
- $p(x_t \mid u_t, x_{t-1})$ is the motion model, the probability of each new position given the command and the last position x_{t-1};
- $p(z_t \mid x_t)$ is the sensor model, the probability of each reading given the position.

Each round has two steps:

- **predict** with the command u_t, spreading the belief by the motion model;
- **update** with the reading z_t, multiplying by the sensor model and rescaling to a total of 1.

For a general belief these steps act on whole curves, which is costly: the **particle filter** (G-2492) needs hundreds of samples to store one. The **Kalman filter** (G-2506) instead uses the case in which the belief is always a Gaussian, which needs no samples at all.

2.2 A Gaussian is two numbers

A Gaussian, also called a normal distribution (G-827), is fixed by its mean (where the peak is) and its **variance** (G-2076) (how wide it is). If the belief is normal, it is enough to store:

- $\hat{x}$ is the estimate, the mean, our best single guess, in m;
- p is its variance, how unsure we are, in m^2.

The robot's first guess, marked with the subscript 0 for the start, is that it starts at the dock, give or take about 1 m:

$$\hat{x}_0 = 0 \text{ m}$$

$$p_0 = 1 \text{ m}^2$$

The true start is 0.3 m; the filter does not know that, and this start error is part of the commands' drift. Two numbers are enough only if the predict and update steps keep the **belief** (G-2433) Gaussian, which the rest of this section checks.

2.3 A linear function keeps a Gaussian a Gaussian

> **Key point:** Passing a normal variable x through a linear function $y = ax + b$ (a fixed multiple a of x plus a constant b), whose graph is a straight line, gives another normal variable y: the mean goes through the line and the variance is multiplied by a^2, the slope squared.

The predict step shifts the **belief** (G-2433) by the command, so we first check what a linear function does to a Gaussian, on a small example:

- x is a normal variable with mean 1 and **standard deviation** (G-1871), the typical distance from the mean, 0.5;
- y is the value the line below gives for x.

$$y = 2x + 1$$

Every value of x is stretched by 2 and shifted by 1, so the whole curve is stretched by 2 and shifted by 1. Its shape stays a **Gaussian** (G-827) (Figure 3). The mean goes through the line:

$$\text{mean of } y = 2 \times 1 + 1 = 3$$

The standard deviation is stretched by the slope:

$$\text{standard deviation of } y = 2 \times 0.5 = 1$$

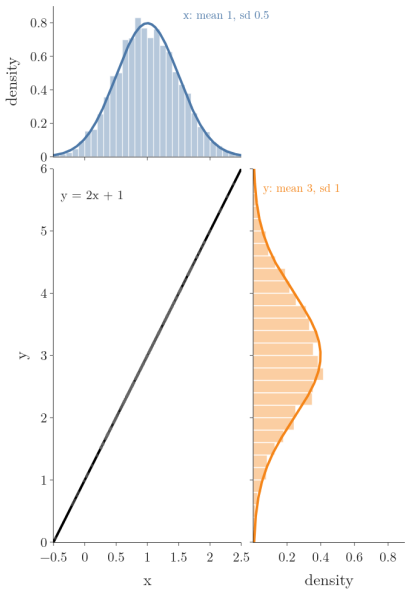


Figure 3: x with mean 1 and standard deviation 0.5 (top) is passed through the line y = 2x + 1; the result y (right) is a Gaussian with mean 3 and standard deviation 1. Both axes and both density panels use the same scale, so the line rises 2 for every 1 across and the curve of y is twice as wide and half as tall. Histograms of 5000 samples sit on the formula curves

Why the mean goes through the line and the variance gets a^2, the slope squared. The mean of y is the average of all the y values, and every y is $2x + 1$. Two samples of x, 0.5 and 1.5, have mean 1 and variance 0.25, like x in Figure 3. Through the line they become 2 and 4:

$$\text{mean of } y = \frac{2+4}{2}$$

$$= \frac{(2 \times 0.5 + 1) + (2 \times 1.5 + 1)}{2}$$

$$= \frac{2 \times 0.5 + 2 \times 1.5 + 1 + 1}{2}$$

$$= \frac{2 \times (0.5 + 1.5) + 2 \times 1}{2}$$

$$= 2 \times \frac{0.5 + 1.5}{2} + 1$$

$$= 2 \times 1 + 1$$

$$= 3$$

The same steps for any samples, through any line:

- n is the number of samples, 2 above;
- x_i is sample number i, 0.5 and 1.5 above;
- μ is the mean of the samples, 1 above;
- a is the line's slope, 2 above;
- b is the line's shift, 1 above;
- y_i is sample x_i after the line, 2 and 4 above;
- $\sum$ means add up, here over i from 1 to n.

$$\text{mean of } y = \frac{1}{n}\sum_{i=1}^{n} y_i$$

$$= \frac{1}{n}\sum_{i=1}^{n}(ax_i + b)$$

$$= \frac{1}{n}\left(\sum_{i=1}^{n} ax_i + \sum_{i=1}^{n} b\right)$$

$$= \frac{1}{n}\left(a\sum_{i=1}^{n} x_i + nb\right)$$

$$= a\,\frac{1}{n}\sum_{i=1}^{n} x_i + b$$

$$= a\mu + b$$

The variance (G-2076) of y, written $\mathrm{Var}(y)$, is the average squared distance of the y values from their mean, $a\mu + b$:

$$\mathrm{Var}(y) = \frac{1}{n}\sum_{i=1}^{n}\Big(y_i - (a\mu + b)\Big)^2$$

$$= \frac{1}{n}\sum_{i=1}^{n}(ax_i + b - a\mu - b)^2$$

$$= \frac{1}{n}\sum_{i=1}^{n}(ax_i - a\mu)^2$$

$$= \frac{1}{n} \sum_{i=1}^{n} \Big(a \left(x_i - \mu \right) \Big)^2$$

$$= \frac{1}{n} \sum_{i=1}^{n} a^2 \left(x_i - \mu \right)^2$$

$$= a^2 \frac{1}{n} \sum_{i=1}^{n} (x_i - \mu)^2$$

$$= a^2 \operatorname{Var}(x)$$

With our numbers:

$$\operatorname{Var}(y) = 4 \times 0.25 = 1$$

The shift b moves the Gaussian without widening it. That the result is still a Gaussian, not some other shape, is a property of the normal distribution (Freiburg KF slides; Labbe ch.4). The command is such a shift, but each step also adds a wheel slip, a second error.

2.4 Adding independent errors adds variances

The robot's next position is its present position plus the commanded 0.5 m plus a random slip. The estimate moves by the same 0.5 m. The error is the true position minus the estimate. With:

- x is the robot's present true position, in m;
- $\hat{x}$ is the present estimate, in m;
- w is the next step's wheel slip, in m, with mean 0 and a standard deviation of 0.1 m;
- e is the present error, the true position minus the estimate, in m.

The next error:

$$(x + 0.5 + w) - (\hat{x} + 0.5)$$

$$= x - \hat{x} + w$$

$$= e + w$$

The present error has mean 0, because the estimate is the mean, and **variance** (G-2076) p, the estimate's variance, 1 m^2 at the start (Section 2.2). The slip has mean 0 and does not depend on e, so the two errors are independent. Their variances add, where Var stands for the variance:

$$\mathrm{Var}(e + w) = \mathrm{Var}(e) + \mathrm{Var}(w)$$

Why. Take a present error of 1 m or -1 m and a slip of 0.1 m or -0.1 m, all four pairs equally likely. Square each next error:

$$(1 + 0.1)^2 = 1 + 0.2 + 0.01$$

$$(1 - 0.1)^2 = 1 - 0.2 + 0.01$$

$$(-1 + 0.1)^2 = 1 - 0.2 + 0.01$$

$$(-1 - 0.1)^2 = 1 + 0.2 + 0.01$$

Average the four squares; each $+0.2$ cancels a -0.2:

$$\mathrm{Var}(e + w) = 1 + 0.01$$

The same in symbols:

$$(e + w)^2 = e^2 + 2ew + w^2$$

Averaged, the $2ew$ parts cancel the same way: whatever e is, the slip is as often negative as positive.

$$\mathrm{Var}(e + w) = \mathrm{Var}(e) + \mathrm{Var}(w)$$

So every slip adds its variance, and the commands' drift grows step by step.

2.5 The linear Gaussian system

Sections 2.3 and 2.4 give the two facts the Kalman filter needs: a linear function keeps a Gaussian a Gaussian, and independent errors add their **variances** (G-2076). Both apply to a **linear Gaussian system** (G-2507), a system that meets three conditions (Freiburg KF slides; Thrun §3.2):

1. the next state is a linear function of the state and the command, plus normal noise;
2. the reading is a linear function of the state, plus normal noise;
3. the first **belief** (G-2433), before any command or reading, is normal.

Our robot meets all three. At step t:

- x_t is the robot's true position, in m;
- u_t is the command, 0.5 m forward;
- w_t is the wheel slip, the source of the commands' drift, normal with mean 0 and variance $Q = 0.01$ m^2 (a standard deviation of 0.1 m);
- z_t is the beacon's reading, in m;
- v_t is the beacon's error, the source of its jumps, normal with mean 0 and variance $R = 0.16$ m^2 (a standard deviation of 0.4 m).

The next position is the last position plus the command plus the slip:

$$x_t = x_{t-1} + u_t + w_t$$

The reading is the position plus the beacon's error:

$$z_t = x_t + v_t$$

Both are linear with normal noise, which meets conditions 1 and 2. The first belief, mean 0 m and variance 1 m^2 (Section 2.2), is normal, which meets condition 3. So the belief stays Gaussian at every step, and the Kalman filter computes its mean and variance exactly. It is the **Bayes filter** (G-2446) with nothing approximated (Freiburg KF slides). Its first step, predict, is where the drift enters.

3. Predict: the Gaussian moves and widens

> **Key point:** The command moves the estimate; the slip widens it. The predicted **variance** (G-2076), how unsure the prediction is, is the old variance plus the slip's variance Q, 0.01 m^2.

The idea. First the drift of Figure 1: why do the commands alone miss by 0.27 m? The robot is commanded 0.5 m, so our best guess moves 0.5 m. But each command adds a slip we cannot see, so we are less sure than before, and the doubt piles up.

The standard terms. The random slip the motion adds at each step is the **process noise** (G-2509) (kalmanfilter.net). The estimate before the step's reading is the **predicted estimate** (G-2512), also called the prior or a priori estimate.

The formulas. At each step:

- $\hat{x}$ is the estimate after the last step, in m (0 m at the start);
- p is its variance, in m^2 (1 m^2 at the start);
- u is the command, 0.5 m;
- Q is the slip's variance, 0.01 m^2;
- $\bar{x}$ is the predicted estimate, in m;
- $\bar{p}$ is its variance, in m^2.

The mean goes through the motion; the variances add (Section 2.4):

$$\bar{x} = \hat{x} + u$$

$$\bar{p} = p + Q$$

Step 1 on the corridor. From the first guess:

$$\bar{x}_1 = 0 + 0.5 = 0.5 \text{ m}$$

$$\bar{p}_1 = 1 + 0.01 = 1.01 \text{ m}^2$$

Frame "step 1: predict" of Figure 7 (Section 5) shows the **Gaussian** (G-827), the bell-shaped belief, moving 0.5 m and widening a little.

If the robot drove with no readings at all, the variance would grow by Q every step. After 100 steps:

$$p = 1 + 100 \times 0.01 = 2 \text{ m}^2$$

$$\sqrt{2} = 1.4 \text{ m standard deviation}$$

The growing variance is the drift of Figure 1: the commands alone only ever widen the Gaussian, so a beacon reading is needed to pull the variance down.

4. Update: the gain that makes the estimate most certain

> **Key point:** The new estimate is a weighted average of the prediction and the reading. The weight on the reading that gives the smallest **variance** (G-2076), the spread of the error, is the Kalman gain K: the prediction's variance $\bar{p}$ divided by the sum of $\bar{p}$ and the reading's variance R, so it is large when the prediction is unsure and small when the reading is.

4.1 Two estimates of one position

The beacon stops the drift but jumps. The update decides how far to follow each reading. After the predict step of step 1, we have two estimates of the same position:

Source	Value	Variance
prediction	$\bar{x}_1 = 0.5$ m	$\bar{p}_1 = 1.01$ m^2
beacon reading	$z_1 = 0.82$ m	$R = 0.16$ m^2

The reading's error has a **variance** (G-2076) R (its spread), the **measurement noise** (G-2510) variance, which comes from the sensor's datasheet or from a calibration (Section 7.1).

Neither should be thrown away. A natural blend is a weighted average:

- z is the beacon's reading, in m (0.82 m at step 1);
- $\bar{x}$ is the **predicted estimate** (G-2512), in m (0.5 m at step 1);
- k is the weight on the reading, a number from 0 to 1; the prediction gets $1 - k$;
- $\hat{x}$ is the blended estimate, in m.

$$\hat{x} = k\,z + (1 - k)\,\bar{x}$$

Which k? Taking $k = 0$ keeps the prediction, $k = 1$ takes the reading, and anything in between mixes them. We want the k that makes the blended estimate as certain as possible: the smallest variance.

4.2 The variance of the blend

To find that k, we write the blend's **variance** (G-2076) as a function of k. The prediction's error and the reading's error are independent. By Section 2.3, scaling an error by k scales its variance by k^2; by Section 2.4, independent variances add:

- $p(k)$ is the variance of the blend for weight k, in m^2;
- R is the reading's variance, 0.16 m^2;
- $\bar{p}$ is the prediction's variance, 1.01 m^2 at step 1.

$$p(k) = k^2 R + (1 - k)^2\,\bar{p}$$

With our numbers:

$$p(k) = 0.16\,k^2 + 1.01\,(1 - k)^2$$

Figure 4 slides the weight k from 0 to 1 and plots the blend's variance $p(k)$. At the ends:

$$p(0) = 1.01 \quad \text{(prediction only)}$$

$$p(1) = 0.16 \quad \text{(reading only)}$$

In between, the dot dips below both ends, and there the blended Gaussian is narrower than either of the two: mixing two independent estimates beats either one. What is left is to find where the dip is lowest.

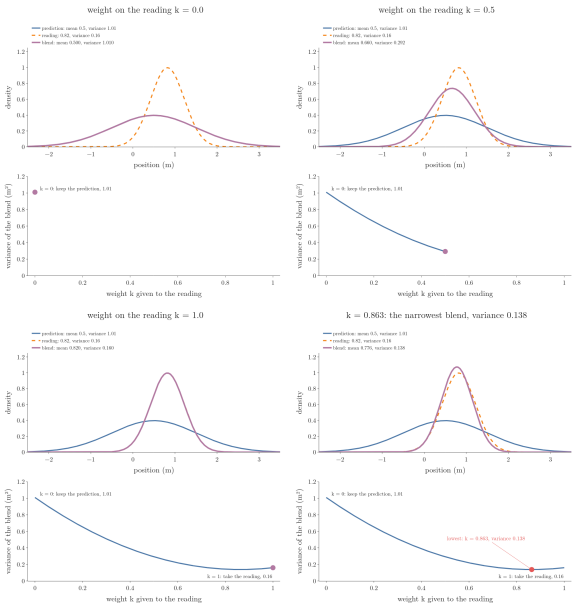


Figure 4: Step 1's prediction (mean 0.5 m, variance 1.01 m^2) and reading (0.82 m, variance 0.16 m^2) blended with the weight k on the reading, k sliding from 0 to 1, one value per frame. Top: the blended Gaussian moves from the prediction to the reading. Bottom: a dot traces the blend's variance p(k), 1.01 at k = 0 and 0.16 at k = 1. The last frame shows the lowest point, k = 0.863 with variance 0.138, where the blend is narrowest

4.3 The best weight: the Kalman gain

The lowest point of a smooth curve is where its slope is zero. The symbols are those of Section 4.2: the weight k on the reading, the blend's variance $p(k)$, the reading's variance R and the prediction's variance $\bar{p}$. One more:

- dp/dk is the **derivative** (G-595) of $p(k)$: its slope, how fast the blend's variance changes as the weight k grows, in m^2 per unit of weight.

Term by term:

$$\frac{dp}{dk} = 2kR - 2(1-k)\,\bar{p}$$

Set it to zero and divide by 2:

$$kR = (1-k)\,\bar{p}$$

$$kR + k\bar{p} = \bar{p}$$

$$k = \frac{\bar{p}}{\bar{p} + R}$$

This best weight is the **Kalman gain** (G-2508), written K (kalmanfilter.net; Becker ch.4). For step 1:

$$K_1 = \frac{1.01}{1.01 + 0.16}$$

$$= 1.01/1.17 = 0.863$$

The reading gets 86 percent of the say, because our first guess (**variance** (G-2076) 1.01) was far less certain than the beacon (0.16).

Checks at easy values (kalmanfilter.net):

- A perfect sensor, $R = 0$: the top and bottom of the gain are equal, so $K = 1$, and the estimate is the reading.
- A perfect prediction, $\bar{p} = 0$: $K = 0$, and the reading is ignored.
- Equal variances: $K = 0.5$, a plain average.

So K always lies between 0 and 1, and the update always lands between the prediction and the reading.

4.4 The updated estimate and its variance

Put the gain K in place of the weight k in the weighted average of Section 4.1 and regroup. At step 1:

- z is the beacon's reading, 0.82 m;
- $\bar{x}$ is the predicted estimate, 0.5 m;
- K is the Kalman gain, the weight on the reading, 0.863;
- $\hat{x}$ is the updated estimate, in m.

$$\hat{x} = Kz + (1 - K)\,\bar{x}$$

$$\hat{x} = \bar{x} + K\,(z - \bar{x})$$

The second form reads: start from the prediction and move a fraction K of the way towards the reading. The gap between the reading and the prediction, $z - \bar{x}$, is the **innovation** (G-2511): the part of the reading the prediction did not expect.

For step 1, the innovation is:

$$z_1 - \bar{x}_1 = 0.82 - 0.5 = 0.32$$

The updated estimate:

$$\hat{x}_1 = 0.5 + 0.863 \times 0.32$$

$$= 0.5 + 0.276 = 0.776 \text{ m}$$

The true position is 0.73 m.

The variance. Put $k = K$ into the blend's variance of Section 4.2. At step 1:

- $p(k)$ is the blend's variance for weight k, in m^2;
- $\bar{p}$ is the prediction's variance, 1.01 m^2;
- R is the reading's variance, 0.16 m^2;
- p is the updated estimate's variance, in m^2.

The two weights are:

$$K = \frac{\bar{p}}{\bar{p} + R}$$

$$1 - K = \frac{R}{\bar{p} + R}$$

So the two terms of $p(k)$ are:

$$K^2 R = \frac{\bar{p}^2 R}{(\bar{p} + R)^2}$$

$$(1 - K)^2 \, \bar{p} = \frac{R^2 \, \bar{p}}{(\bar{p} + R)^2}$$

Add them and take out the shared $\bar{p}R$:

$$p = \frac{\bar{p}R \, (\bar{p} + R)}{(\bar{p} + R)^2}$$

$$= \frac{R}{\bar{p} + R} \, \bar{p}$$

$$= (1 - K) \, \bar{p}$$

(kalmanfilter.net).

For step 1:

$$p_1 = 0.137 \times 1.01 = 0.138 \text{ m}^2$$

the lowest point of Figure 4. The estimate after the reading is the **updated estimate** (G-2513), also called the posterior or a posteriori estimate. Because $1 - K$ is at most 1, an update never makes the filter less sure: one beacon reading has cut the **standard deviation** (G-1871), the typical size of the error, from 1 m to 0.37 m.

4.5 The same answer from multiplying Gaussians

The gain came from making the **variance** (G-2076) smallest, not from the Bayes filter. The Bayes filter's update multiplies the predicted belief by the sensor model and rescales, as the continuous Bayes filter does. Both are **Gaussians** (G-827), bell curves, here. At step 1:

- $\bar{x}$ is the predicted estimate, the mean of the predicted belief, 0.5 m;
- $\bar{p}$ is its variance, 1.01 m^2;
- z is the beacon's reading, the centre of the sensor model's Gaussian, 0.82 m;
- R is the reading's variance, which sets the sensor model's width, 0.16 m^2.

The product of two Gaussians is again a Gaussian (Labbe ch.4; Freiburg KF slides), with mean and variance:

$$\text{mean} = \frac{R\,\bar{x} + \bar{p}\,z}{\bar{p} + R}$$

$$\text{variance} = \frac{\bar{p}\,R}{\bar{p} + R}$$

For step 1:

$$\text{mean} = \frac{0.16 \times 0.5 + 1.01 \times 0.82}{1.17}$$

$$= 0.908/1.17 = 0.776$$

$$\text{variance} = \frac{1.01 \times 0.16}{1.17} = 0.138$$

The same numbers as Section 4.4: the variance-minimising blend and the Bayes update are one and the same. Figure 5 multiplies the step-1 prediction by readings of different quality. A vague reading ($R = 4$) gives the gain $K = 0.202$ and barely moves the Gaussian; the real beacon ($R = 0.16$) gives $K = 0.863$; a sharp reading ($R = 0.01$) gives $K = 0.990$ and the product sits on the reading. So the **measurement noise** (G-2510) R sets how far one update follows a reading.

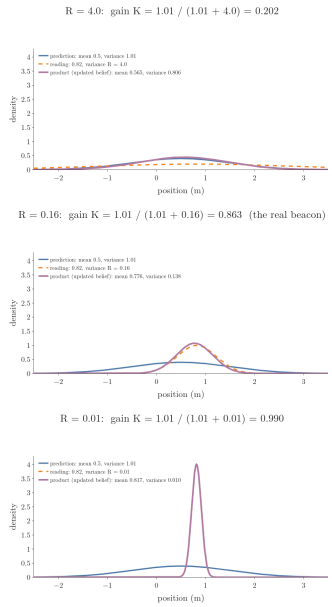


Figure 5: The step-1 prediction (mean 0.5, variance 1.01) multiplied by a reading at 0.82 whose variance R falls from 4 to 0.01: the product moves from the prediction to the reading and narrows; at R = 0.16 the product has mean 0.776 and variance 0.138

5. Ten steps on the corridor

> **Key point:** Predict, then update, every step. The gain starts high, while the first guess is vague, and settles to a fixed value once the **variance** (G-2076), the filter's uncertainty, stops changing.

One step has cut the variance from 1.01 to 0.138 m^2. The ten steps of Figure 1 are this loop run ten times: predict with the drifting command, update with the jumping reading (Figure 6). The filter keeps only the estimate $\hat{x}$ and its variance p from one step to the next: it never stores the past readings, yet its estimate uses all of them (kalmanfilter.net; Labbe ch.4).

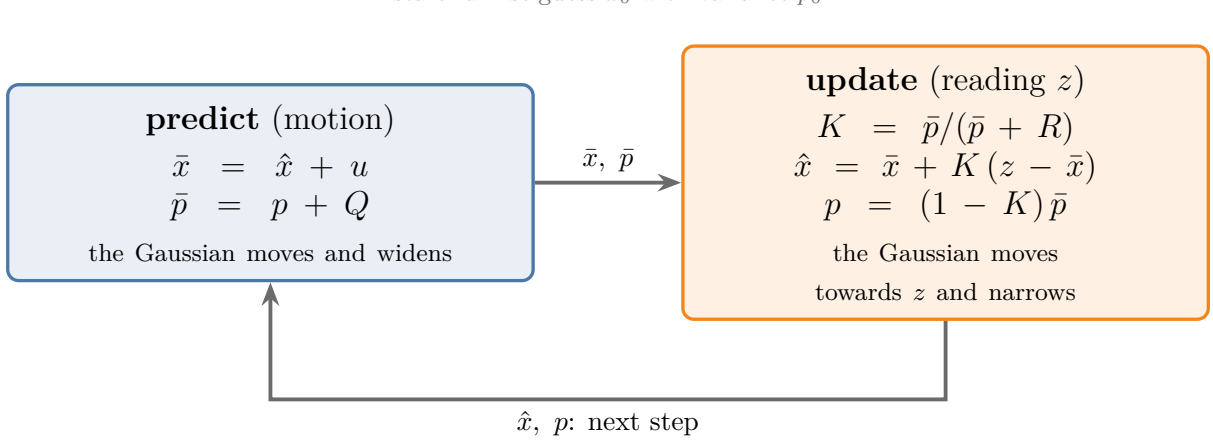


Figure 6: The one-dimensional Kalman filter as a loop: predict moves the mean by the command u and adds the slip's variance Q to the variance; update computes the Kalman gain K, moves the mean a fraction K towards the reading and multiplies the variance by 1 − K

Step 1 is Sections 3 and 4. Its numbers, with the step number as a subscript:

- $\bar{x}_1$ is the predicted estimate, 0.5 m;
- $\bar{p}_1$ is its variance, 1.01 m^2;
- z_1 is the beacon's reading, 0.82 m;
- K_1 is the Kalman gain, the weight on the reading, 0.863;
- $\hat{x}_1$ is the updated estimate, 0.776 m;
- p_1 is its variance, 0.138 m^2.

Step 2. Predict from step 1's estimate, adding the command u, 0.5 m, and the slip's variance Q, 0.01 m^2:

$$\bar{x}_2 = 0.776 + 0.5 = 1.276$$

$$\bar{p}_2 = 0.138 + 0.01 = 0.148$$

The reading is $z_2 = 1.85$ m. The gain, with the beacon's variance R, 0.16 m^2:

$$K_2 = 0.148/(0.148 + 0.16)$$

$$= 0.148/0.308 = 0.481$$

The prediction is now about as certain as the reading, so the gain is near one half. The **innovation** (G-2511), the reading minus the prediction:

$$z_2 - \bar{x}_2 = 1.85 - 1.276 = 0.574$$

The update:

$$\hat{x}_2 = 1.276 + 0.481 \times 0.574$$

$$= 1.276 + 0.276 = 1.552 \text{ m}$$

$$p_2 = 0.519 \times 0.148 = 0.077 \text{ m}^2$$

The true position is 1.22 m. The reading 1.85 m was 0.63 m too high, and the filter followed it only halfway.

Steps 3 to 10 (notebook):

Step	$\bar{x}$	$\bar{p}$	K	z	$\hat{x}$	p	True
1	0.500	1.010	0.863	0.82	0.776	0.138	0.73
2	1.276	0.148	0.481	1.85	1.552	0.077	1.22
3	2.052	0.087	0.352	2.01	2.037	0.056	1.88
4	2.537	0.066	0.293	2.65	2.570	0.047	2.45
5	3.070	0.057	0.262	2.19	2.839	0.042	2.79
6	3.339	0.052	0.245	4.18	3.545	0.039	3.28
7	4.045	0.049	0.235	2.95	3.788	0.038	3.72
8	4.288	0.048	0.229	4.68	4.378	0.037	4.24
9	4.878	0.047	0.226	4.45	4.781	0.036	4.58
10	5.281	0.046	0.224	4.75	5.162	0.036	5.10

Variances are in m^2, positions in m.

Figure 7 plays the table, one stage per frame. Watch the gain dot at the bottom fall and settle, and frame “step 6: update”, where a reading 0.90 m too high moves the estimate only 0.21 m.

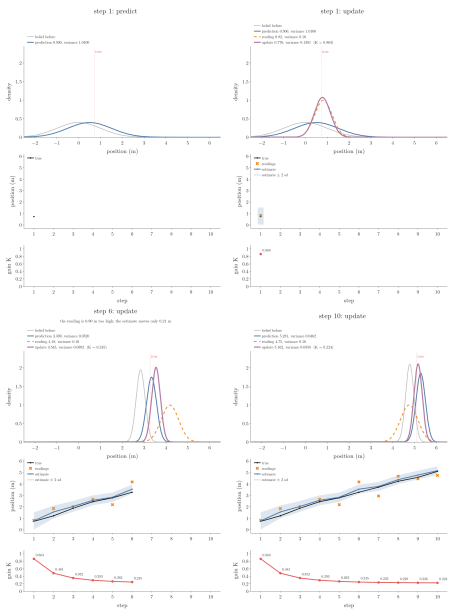


Figure 7: The ten steps of the table, one stage per frame. Top: the predicted Gaussian moves 0.5 m and widens, the reading's Gaussian arrives, and the updated Gaussian lies between them and is narrower; the dotted line is the true position. Middle: true position, readings and the estimate with a band of two standard deviations, added step by step. Bottom: the gain K, the weight on the reading, one dot per update. The last frame shows all ten steps: the gain has fallen from 0.863 at step 1 to 0.224 at step 10

What to see in Figures 6 and 7:

- **The gain falls, then settles.** Each update shrinks the variance, and each prediction adds back only the slip's variance Q, 0.01 m^2. After a few steps the two balance: the variance before each reading settles near 0.046 and the gain near 0.22 (0.2207 in the long run, notebook). The filter has stopped learning about its start and now trusts each reading about a fifth. The fixed value the gain approaches is the **steady-state gain** (G-2518).
- **The jumps are damped.** The beacon's step-6 reading jumped 0.9 m too high; the estimate moved only 0.21 m towards it, because the gain was 0.245.
- **The band is consistent.** The shaded band in the middle panel of Figure 7 is the estimate plus and minus two **standard deviations** (G-1871), twice its typical error; the true position stays inside it at every step, a first sign that the filter's variance matches its real errors, which is what makes a **consistent** filter (G-2524).

Over the ten steps, the commands alone miss by 0.27 m, the beacon by 0.51 m, the filter by 0.17 m (notebook). Blending the two sources this way solves the opening problem for a state of one number, the position, moved by a known command.

6. The matrix form: more than one number

> **Key point:** With a state of several numbers, the **variance** (G-2076), one number for the spread, becomes a covariance matrix and the same five lines become matrix lines. The covariance between position and speed lets a position reading correct a speed that is never measured.

6.1 Why a vector state: the hidden speed

So far the commands told the filter how far the robot moved, 0.5 m per step, drift and all. Suppose instead we do not know the robot's speed, only that it drives roughly steadily: a person is pushing a trolley, or the odometry is unreliable. We still want the speed, for example because a speed controller needs it (Section 6.6), but nothing measures it.

We put the speed into the state. The state is now a vector (a list of numbers), written in bold:

- $\mathbf{x}$ is the state, a vector of 2 entries: the position, in m, on top, and the speed, in m/s, below.

$$\mathbf{x} = \begin{bmatrix} \text{position} \\ \text{speed} \end{bmatrix}$$

Its uncertainty is a 2×2 covariance matrix (G-495). The belief is a multivariate normal distribution (G-1283), the **Gaussian** (G-827) for several numbers at once, with:

- $\hat{\mathbf{x}}$ is its mean, the estimate, a vector of 2 entries: the estimated position, in m, and speed, in m/s (0 and 0 at the start, Section 6.4);
- P is its covariance matrix, 2×2: the **variances** (G-2076) of position, in m^2, and speed, in m^2/s^2, on the diagonal, and their covariance, in m^2/s, off it (1, 1 and 0 at the start).

The motion and the reading must now act on this vector, as matrices.

6.2 The model as matrices

Motion. With steps of 1 s, the position grows by the speed and the speed stays as it is, plus noise. As a matrix times the state, at step t:

- $\mathbf{x}_t$ is the true state, a vector of 2 entries: position, in m, and speed, in m/s;
- F is a 2×2 matrix that carries the state one step forward, given below;
- $\mathbf{w}_t$ is the process noise, a vector of 2 entries: the position slip, in m, and the speed change, in m/s, each with mean 0; their covariance matrix Q is given below.

$$\mathbf{x}_t = F\,\mathbf{x}_{t-1} + \mathbf{w}_t$$

$$F = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

The first row gives position plus speed; the second gives speed. F is the **state transition matrix** (G-2514); this choice of F is the constant-velocity model (Labbe ch.6). The noise $\mathbf{w}_t$ has covariance Q, a 2×2 matrix: the variance of the position slip, in m^2, and of the speed change, in m^2/s^2, on the diagonal, and 0 off it, as the two are not linked:

$$Q = \begin{bmatrix} 0.01 & 0 \\ 0 & 0.001 \end{bmatrix}$$

a position slip of 0.1 m and a speed change of about 0.03 m/s per step (standard deviations).

Reading. The beacon reads position, not speed. At step t:

- z_t is the beacon's reading, one number, in m;
- H is a 1×2 matrix, a row of two numbers, that turns the state into the reading the sensor should give, given below;
- v_t is the beacon's error, normal with mean 0 and variance R, 0.16 m^2.

$$z_t = H\,\mathbf{x}_t + v_t$$

$$H = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

H is the **measurement matrix** (G-2515): it picks out of the state what the sensor sees, here the position. The **measurement noise** (G-2510) is the same, $R = 0.16$ m^2. With F and H in hand, the filter's lines can be rewritten for matrices.

6.3 The five lines in matrix form

Each line is the one-dimensional line with numbers replaced by matrices (Labbe ch.6; kalman-filter.net; Freiburg KF slides). The one-number column uses the symbols of Section 3 and Section 4. The matrix column uses those of Section 6.1 and Section 6.2: the estimate $\hat{\mathbf{x}}$ and its covariance matrix P, the state transition matrix F, the process noise covariance Q, the measurement matrix H, the reading z and the beacon's variance R, 0.16 m^2. New here:

- F^{T} is the transpose (G-2012) of F: F flipped over its diagonal, 2×2;
- $\bar{\mathbf{x}}$ is the predicted estimate, a vector of 2 entries: position, in m, and speed, in m/s;
- $\bar{P}$ is its covariance matrix, 2×2;
- H^{T} is the transpose of H, a column of 2 entries;
- S is the innovation variance, the gain's denominator, which gets its own row and name; here one number, in m^2;
- S^{-1} is the inverse (G-968) of S, here simply 1 divided by the number S;
- K is the Kalman gain, here a vector of 2 entries: one weight for position and one for speed;
- I is the identity matrix, 2×2, with 1 on the diagonal and 0 off it: multiplying by it changes nothing.

Step	One number	Matrix form
predict mean	$\bar{x} = \hat{x} + u$	$\bar{\mathbf{x}} = F\,\hat{\mathbf{x}}$
predict variance	$\bar{p} = p + Q$	$\bar{P} = FPF^{\mathsf{T}} + Q$
innovation variance	$\bar{p} + R$	$S = H\bar{P}H^{\mathsf{T}} + R$
gain	$K = \bar{p}/(\bar{p} + R)$	$K = \bar{P}H^{\mathsf{T}}S^{-1}$
update mean	$\hat{x} = \bar{x} + K(z - \bar{x})$	$\hat{\mathbf{x}} = \bar{\mathbf{x}} + K(z - H\bar{\mathbf{x}})$
update variance	$p = (1 - K)\,\bar{p}$	$P = (I - KH)\,\bar{P}$

The only new piece is FPF^{T}, the matrix version of Section 2.3's rule "**variance** (G-2076) times slope squared": a linear map F turns a covariance P into FPF^{T} (Labbe ch.6). The matrix gain is derived the same way as in Section 4.3, by making the updated variances as small as possible (Labbe ch.7, "Deriving the **Kalman gain** (G-2508) equation").

6.4 One step in matrix form, and the speed nobody measured

We run one step of these lines on the corridor's first reading. At the start, marked with the subscript 0:

- $\hat{\mathbf{x}}_0$ is the first estimate, a vector of 2 entries: position 0 m and speed 0 m/s;
- P_0 is its covariance matrix, 2×2: **variance** (G-2076) 1 m^2 for position and 1 $\mathrm{m}^2/\mathrm{s}^2$ for speed on the diagonal, and no covariance, 0, off it;
- F is the state transition matrix of Section 6.2, with rows (1, 1) and (0, 1);
- Q is the process noise covariance of Section 6.2, 0.01 m^2 and 0.001 $\mathrm{m}^2/\mathrm{s}^2$ on the diagonal.

$$\hat{\mathbf{x}}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$P_0 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Predict. The mean stays at zero. P_0 is the identity matrix, so FP_0F^{T} is FF^{T}. Each entry is a row of F times a row of F:

$$\text{top left: } 1 \times 1 + 1 \times 1 = 2$$

$$\text{off-diagonal: } 1 \times 0 + 1 \times 1 = 1$$

$$\text{bottom right: } 0 \times 0 + 1 \times 1 = 1$$

$$FP_0F^{\mathsf{T}} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$$

Adding Q gives the predicted covariance matrix for step 1:

$$\bar{P}_1 = \begin{bmatrix} 2.01 & 1 \\ 1 & 1.001 \end{bmatrix}$$

The position variance has grown from 1 to 2.01, because an unknown speed adds an unknown distance. The off-diagonal 1 is new: the prediction now links the two errors. If the robot is further along than predicted, it was probably faster.

Update with the first reading $z_1 = 0.82$ m. The innovation variance S is the predicted position variance plus the beacon's variance R, 0.16 m^2:

$$S = 2.01 + 0.16 = 2.17$$

The measurement matrix H, the row (1, 0), picks the first column of $\bar{P}_1$:

$$\bar{P}_1 H^\mathsf{T} = \begin{bmatrix} 2.01 \\ 1 \end{bmatrix}$$

Dividing by S gives the gain K, one weight for position and one for speed:

$$K = \begin{bmatrix} 2.01/2.17 \\ 1/2.17 \end{bmatrix}$$

$$= \begin{bmatrix} 0.9263 \\ 0.4608 \end{bmatrix}$$

The prediction was 0, so the **innovation** (G-2511), the reading minus the predicted reading, is the whole reading, 0.82 m. The new state:

$$\text{position: } 0.9263 \times 0.82 = 0.760$$

$$\text{speed: } 0.4608 \times 0.82 = 0.378$$

$$\hat{\mathbf{x}}_1 = \begin{bmatrix} 0.760 \\ 0.378 \end{bmatrix}$$

One position reading has given a speed estimate of 0.378 m/s. The gain's second entry is the covariance divided by S: through the link in $\bar{P}$, a position innovation becomes a speed correction.

Figure 8 feeds the filter the same ten jumping beacon readings, one per frame; watch the speed panel, which no sensor reads. The speed estimate swings at first, then settles: after step

10 it is 0.421 m/s with a standard deviation of 0.081 m/s, against the robot's true average speed of 0.48 m/s, inside one **standard deviation** (G-1871), the filter's own typical error (notebook). A quantity that no sensor measures has been estimated from how the measured one changes, which kalmanfilter.net and Labbe ch.6 call the strength of the multivariate filter.

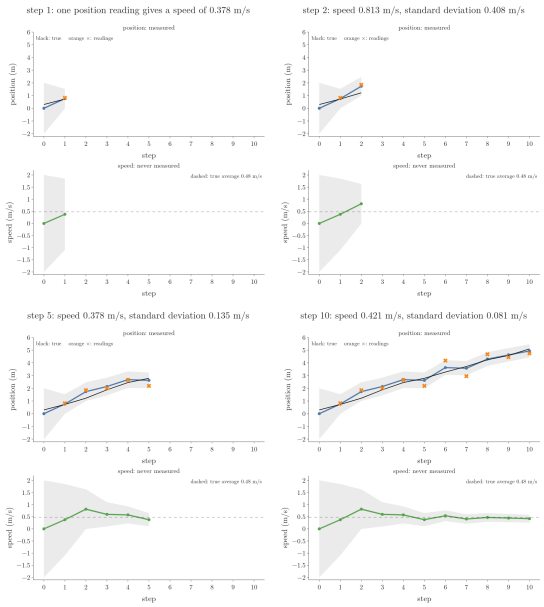


Figure 8: The two-variable filter fed only position readings, one per frame; bands are two standard deviations. Top: position estimate, readings and truth. Bottom: the speed estimate, never measured: one reading gives 0.378 m/s, then it swings and settles. The last frame shows step 10: speed 0.421 m/s with a standard deviation of 0.081 m/s, near the true average of 0.48 m/s, inside a band that has narrowed

6.5 Observability: can the readings pin down the state?

Could we do the same with a speed sensor and no position sensor? No. Every position, shifted by the same amount, gives exactly the same speed readings, so no number of speed readings can tell where the robot is.

A system is **observable** (G-2517) if its whole state can be worked out from its readings (and commands) over time (FBS §8.1). Figure 9 runs two filters for ten steps, one reading position and one reading only speed, one predict or update per frame: each predict raises the variance of the position estimate, each update lowers it. With the speed sensor, the variance after each update is higher than the step before, up to 2.68 m^2 after ten steps; with the position sensor it falls to 0.066 m^2 (notebook). A **Kalman filter** (G-2506) on an unobservable system runs without error messages but cannot shrink the uncertainty in the hidden direction, so checking observability comes before building the filter.

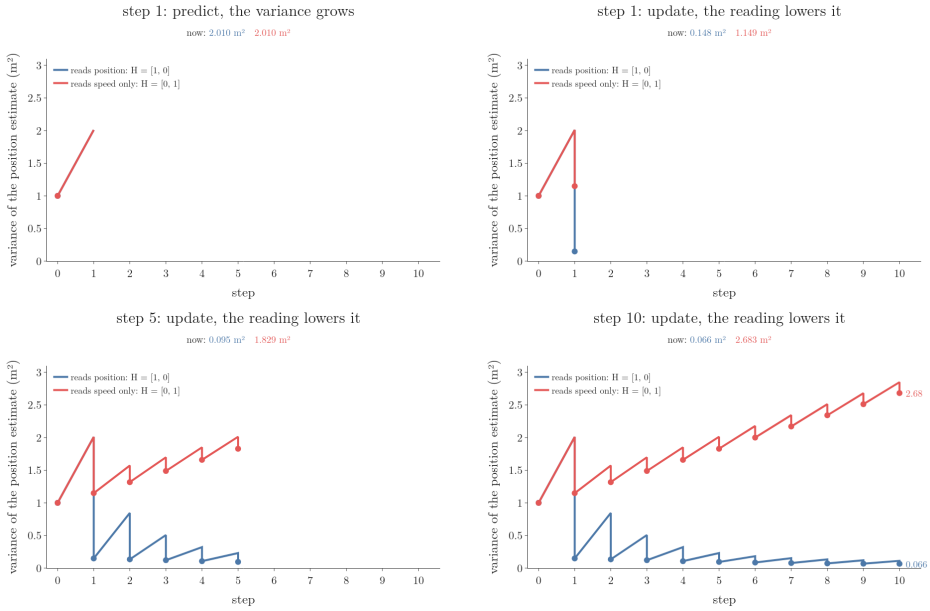


Figure 9: Variance of the position estimate over ten steps, one predict or update per frame: a predict raises it (slanted line), an update lowers it (drop). Blue: the filter reads position (measurement matrix $\mathrm{H} = [1, 0]$); the variance after each update falls every step, to 0.066 m^2. Red: the filter reads only speed ($\mathrm{H} = [0, 1]$); each drop is smaller than the rise before it, so the variance after each update climbs every step, to 2.68 m^2 in the last frame

6.6 The state a controller needs: the filter as an observer

The speed estimate of Section 6.4 is what a controller needs. A controller that holds the robot at 0.5 m/s must know the speed; one that stops it at a door must know the position. Many controllers need the whole state, but sensors give only part of it (FBS §8.1). Something must rebuild the missing parts.

The obvious attempt is to run the model alone, with no readings: each step, multiply the last estimate by the state transition matrix F. Running the model alone is **dead reckoning** (G-2311): any error in the start or in the model is never corrected. The fix is feedback: compare the reading with what the estimate predicts, and correct the estimate by a gain times the gap. With:

- $\bar{\mathbf{x}}$ is the estimate the model predicts, a vector of 2 entries: position, in m, and speed, in m/s;
- z is the beacon's reading, in m;
- H is the measurement matrix of Section 6.2, the row (1, 0), which turns the state into the reading the sensor should give;
- L is the observer's gain, a vector of 2 entries: how far to correct the position and the speed per metre of gap; for the Kalman filter at step 1, 0.926 and 0.461;
- $\hat{\mathbf{x}}$ is the corrected estimate, a vector of 2 entries.

$$\hat{\mathbf{x}} = \bar{\mathbf{x}} + L\,(z - H\bar{\mathbf{x}})$$

A model run alongside the real system and corrected this way is a **state observer** (G-2516) (FBS §8.2). The size of the gain L is a trade-off: a large gain corrects errors fast but passes the sensor's noise straight into the estimate (FBS §8.2). The **Kalman filter** (G-2506) is the observer whose gain, $L = K$, is the best compromise for the given noises (FBS §8.2). The controller then uses $\hat{\mathbf{x}}$ in place of the true state (FBS §8.3), which is why the Kalman filter sits between the sensors and the controller in most robots. Every gain so far rested on two noise settings we were simply given: the slip's variance Q and the beacon's variance R.

7. Choosing Q and R

> **Key point:** The **measurement noise** (G-2510) R is the variance of the beacon's jumps, 0.16 m^2, and is measured; the **process noise** (G-2509) Q is the variance of how far each command can drift from the model, 0.01 m^2, and is tuned. Only their ratio changes the estimates.

7.1 R: measure the sensor

The **measurement noise** (G-2510) R is the variance of the beacon's error, the jumps of Figure 1, so it can be measured: put the robot on a mark at a known position and record many readings (kalmanfilter.net). In a simulated calibration, we parked the robot at exactly 2.00 m and took 100 readings (`data/beacon_calibration.csv`):

$$\text{mean} = 2.018 \text{ m}$$

$$\text{sample variance} = 0.134 \text{ m}^2$$

The datasheet value is 0.16 m^2. Sensor datasheets are the other source of R (kalmanfilter.net). The mean, 2.018 m, also shows no sizeable bias, which matters because the **Kalman filter** (G-2506) assumes the noise has mean zero.

7.2 Q: tune the motion noise

The slip's variance Q is harder: it covers everything the motion model leaves out (slips, pushes, a wheel radius slightly wrong), so there is no datasheet for it, and it is set by tuning against data (Labbe ch.8). Each wrong choice fails in its own way. We simulated 2000 runs of 50 steps in which the true slip **variance** (G-2076) is 0.01, and ran three filters on them, each scored by its RMSE, the typical size of its error (Figure 10):

Filter's Q	Gain it settles to	Error (RMSE, steps 11–50)	What goes wrong
0.0001 (too small)	0.10 at step 10, falling	0.305 m	trusts the commands, ignores readings, drifts with the slips

Filter's Q	Gain it settles to	Error (RMSE, steps 11–50)	What goes wrong
0.01 (right)	0.22	0.187 m	none
1 (too big)	0.88	0.353 m	trusts each reading, copies its noise

The readings alone give 0.399 m. Too small a Q trusts the commands and drifts like them; too large a Q follows every jump of the beacon (Labbe ch.4, "Incorrect process variance"; kalmanfilter.net, where a too-small Q causes a lag error). Figure 10 steps the three filters through one of these runs; watch each line against zero and each filter's gain in the legend.

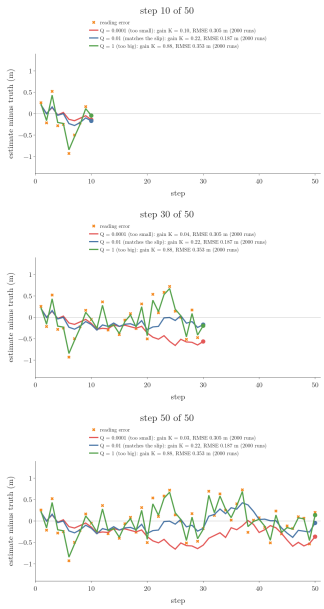


Figure 10: Error of three filters on one 50-step run, one step per frame, each set with a different slip variance Q; each legend entry gives the filter's gain K at the current step and its RMSE over all 2000 runs. Q = 0.0001: gain 0.10 at step 10 and 0.03 at step 50, so its estimate lags below the truth for long stretches; Q = 1: gain 0.88, its estimate jumps with every reading; Q = 0.01: gain 0.22, its estimate stays closest to zero. The last frame shows all 50 steps

Only the ratio matters. Multiply the slip's variance Q, the beacon's variance R and the starting variance all by 10. The first predicted variance $\bar{p}$, the starting variance plus Q, is then 10 times larger too, and so is every later one. So every gain has a 10 on top and below:

$$K = \frac{10\,\bar{p}}{10\,\bar{p} + 10\,R}$$

$$= \frac{\bar{p}}{\bar{p} + R}$$

The gains and the estimates are the same; the notebook finds a largest difference of exactly 0. What a filter designer really chooses is how much the model is trusted relative to the sensor.

How to tell when it is wrong without knowing the truth. On a real robot the truth is unknown, but the **innovations** (G-2511), the gaps between each reading and its prediction, are not: a well-tuned filter's innovations look like random noise of the size its own innovation variance S (Section 6.3) predicts. Testing them is the consistency check on the innovations.

8. Summary

The five lines of the one-dimensional filter, with:

- $\hat{x}$ is the updated estimate, in m (0 m at the start);
- p is its variance, in m^2 (1 m^2 at the start);
- u is the command, 0.5 m;
- Q is the slip's variance, 0.01 m^2;
- $\bar{x}$ is the predicted estimate, in m;
- $\bar{p}$ is its variance, in m^2;
- z is the beacon's reading, in m;
- R is the beacon's variance, 0.16 m^2;
- K is the Kalman gain, the weight on the reading, from 0 to 1.

Line	Formula	Why
predict mean	$\bar{x} = \hat{x} + u$	the command moves the robot
predict variance	$\bar{p} = p + Q$	independent slip variances add
gain	$K = \bar{p}/(\bar{p} + R)$	the weight that makes the blended variance smallest
update mean	$\hat{x} = \bar{x} + K(z - \bar{x})$	move a fraction K of the way to the reading
update variance	$p = (1 - K)\,\bar{p}$	blending two independent estimates beats either

- The corridor robot's two poor sources make one good one: the filter misses by 0.17 m against 0.27 m for the drifting commands and 0.51 m for the jumping beacon, because at every step the gain gives each source the trust its **variance** (G-2076) earns.
- In a linear Gaussian system the **belief** (G-2433) stays Gaussian, because a linear function keeps a Gaussian a Gaussian and independent variances add, so a mean and a variance carry the whole belief.
- The predict step widens the **Gaussian** (G-827) by Q, so the commands alone always drift: after 100 steps the **standard deviation** (G-1871) would be 1.4 m.
- The **Kalman gain** (G-2508) is derived, not chosen: it is the weight on the reading that minimises the updated variance, and it equals the Bayes update of two multiplied Gaussians (0.776 m and 0.138 m^2 at step 1 either way).
- The gain is high while the estimate is vague (0.863 at step 1) and settles near 0.22 once prediction and reading balance, so the beacon's jumps are damped.
- In matrix form the covariance links position and speed, so position readings estimate a speed nobody measures (0.421 m/s against a true 0.48 m/s).
- **Observability** (G-2517) decides what readings can pin down: position readings give both variables, speed readings never give the position, whose variance grows to 2.68 m^2, because every shifted position gives the same speed readings.
- The **Kalman filter** (G-2506) is the observer with the best gain for the given noises, so

it supplies the full state a controller needs from partial sensors.
- R is measured from the sensor, Q is tuned because no datasheet covers what the motion model leaves out; too small a Q drifts like the commands, too large a Q follows every jump, and only the ratio matters.

The robot can now blend drifting commands and jumping readings with two numbers per step. Still missing: the filter needs linear models, and a range beacon's reading is a nonlinear function of the position (its graph is a curve), which the **extended Kalman filter** (G-2520) straightens.

9. Sources

Built from

- MATLAB, "Understanding Kalman Filters, Part 2: State Observers", YouTube, https://www.youtube.com/watch?v=4OerJmPpkRg
- MATLAB, "Understanding Kalman Filters, Part 3: Optimal State Estimator", YouTube, https://www.youtube.com/watch?v=ul3u2yLPwU0
- MATLAB, "Understanding Kalman Filters, Part 4: Optimal State Estimator Algorithm", YouTube, https://www.youtube.com/watch?v=VFXf1lIZ3p8
- Lars Hammarstrand (Chalmers), "4.3.3 Kalman filter tuning and consistency: motion and measurement models", YouTube, https://www.youtube.com/watch?v=WroIS3pPfjM
- Steve Brunton, "Control Bootcamp: Observability", YouTube, https://www.youtube.com/watch?v=iR
- Becker, A. *Kalman Filter from the Ground Up*, and its free tutorial pages: "Kalman Filter in one dimension" https://www.kalmanfilter.net/kalman1d.html, "Adding process noise" https://www.kalmanfilter.net/kalman1d_pn.html, "Multivariate Kalman Filter" https://www.kalmanfilter.net/kalmanmulti.html (kalmanfilter.net 1D, process noise, multivariate; Becker ch.4)
- Labbe, R. *Kalman and Bayesian Filters in Python*, ch.4 "One Dimensional Kalman Filters", ch.6 "Multivariate Kalman Filters", ch.7 "Kalman Filter Math", ch.8 "Designing Kalman Filters", https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python (Labbe ch.4, ch.6, ch.7, ch.8)
- Åström, K. J. and Murray, R. M. (2021). *Feedback Systems*, 2nd ed., ch.8 "Output Feedback", §8.1 Observability, §8.2 State estimation, §8.3 Control using estimated state. Free: https://fbswiki.org (FBS)
- Burgard, W. et al. (Uni Freiburg), *Introduction to Mobile Robotics*, "Bayes Filter – Kalman Filter" slides, http://ais.informatik.uni-freiburg.de/teaching/ss23/robotics/slides/10-kalman.pdf (Freiburg KF slides)

Other references

- NPTEL-NOC IITM, "Introduction to Robotics, #33 Kalman Filter", YouTube, https://www.youtube.com/watch?v=E9QL8XWJIh8
- Thrun, S., Burgard, W. and Fox, D. (2005). *Probabilistic Robotics*. MIT Press. §3.2 "The Kalman Filter"; cited for what the Freiburg slides confirm (Thrun)

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Kalman filter (G-2506)	The Bayes filter for a linear Gaussian system: it keeps the belief as a mean and a (co)variance and updates them with a predict step and an update step, blending a model's prediction with a sensor reading so the result is as certain as possible.
Unimodal (G-2528)	Having a single mode (one peak); a unimodal belief can be held by one Gaussian, so the Kalman filter needs only its mean and variance.
Kalman gain K (G-2508)	The weight the update gives to the reading, $K = \bar{p}/(\bar{p} + R)$ in one dimension: the weight that makes the updated variance smallest, near 1 when the prediction is vague and near 0 when the reading is.
Observability (G-2517)	Whether a system's whole state can be worked out from its readings and commands over time; a filter needs it before it can pin down every state variable.
Linear function (G-2527)	A function $y = ax + b$: a fixed multiple of the input plus a constant, whose graph is a straight line; passing a normal variable through it gives another normal variable, which is why the Kalman filter is exact for linear models.
Linear Gaussian system (G-2507)	A system whose next state and whose readings are linear functions of the state plus normal noise, starting from a normal belief; in it the belief stays normal, so the Kalman filter is exact.
Process noise Q (G-2509)	The random part of the motion the model leaves out (wheel slip, pushes), given as its variance or covariance Q; it widens the belief at every predict step and is usually set by tuning.
Predicted estimate (a priori) (G-2512)	The estimate and its variance after the predict step and before the step's reading, $\bar{x}$ and $\bar{p}$; it says what the model alone expects.

Term	Meaning
Measurement noise R (G-2510)	The random error of a sensor reading, given as its variance or covariance R; it sets how much the filter trusts each reading and is taken from the datasheet or measured in a calibration.
Innovation (G-2511)	The gap between a reading and the reading the prediction expected, $z - H\bar{x}$; the update moves the estimate by the gain times this gap, and its size is used to check the filter.
Updated estimate (a posteriori) (G-2513)	The estimate and its variance after the reading has been blended in, $\hat{x}$ and p; it is the filter's output for the step and the start of the next.
Steady-state gain (G-2518)	The fixed value the Kalman gain settles to once the variance added by each prediction balances the variance removed by each update, such as 0.22 for $Q = 0.01$ and $R = 0.16$.
State transition matrix F (G-2514)	The matrix that carries the state one step forward in a linear model, such as $F = [[1, 1], [0, 1]]$ for position and speed with 1 s steps (position plus speed, speed unchanged).
Measurement matrix H (G-2515)	The matrix that turns the state into the reading the sensor should give, such as $H = [1, 0]$ for a sensor that sees position but not speed.
State observer (G-2516)	A model run alongside the real system and corrected by a gain times the gap between the measured and the predicted reading, so it rebuilds states no sensor measures for a controller to use; the Kalman filter is the observer with the best gain for the given noises.
Particle filter (G-2492)	A Bayes filter that stores the belief as a set of weighted samples (particles) and runs each round as move, weigh, resample; it can hold beliefs of any shape, such as a belief with three modes, one at each of three doors.

Term	Meaning
Belief (G-2433)	The robot's probability for each possible state given all its commands and readings, $\text{bel}(x_t) = p(x_t \mid z_{1:t}, u_{1:t})$; a short summary of the history that says how strongly the evidence points to each state.
Gaussian distribution (G-827)	Another name for the normal distribution: the symmetric, bell-shaped continuous distribution set by its mean and standard deviation, used to model many measurements.
Mode (G-1251)	The value that appears most often in the data, so it sits under the highest peak of a histogram or a distribution curve; it gives a centre for data that are not numbers, and a belief with several peaks has several modes.
Multimodal (G-1275)	Having more than one mode, that is more than one peak (two peaks: bimodal), such as a robot's belief peaking at three doors; it shows the data point to several separate values, so one mean or one best guess would hide them.
Root mean squared error (RMSE) (G-1705)	The square root of the average squared error (MSE), so the error is back in the output's units.
Standard deviation (G-1871)	How far values typically lie from the mean; divide by $n - 1$ for a sample (pandas default) or by n for a population (NumPy default).
Variance of a random variable (G-2076)	How spread out a random variable's values are: the expected squared distance from its expected value, $\text{Var}(X) = E[(X - E[X])^2]$.
Bayes filter (G-2446)	The algorithm that keeps a robot's belief up to date one step at a time: a prediction step after each command and a correction step after each reading; it gives the exact belief without storing the history, under the Markov assumption.
Derivative (G-595)	The slope of a function at a point: how fast the output changes as the input changes there. It is zero at the lowest point of a smooth curve, which is how training finds a minimum.

Term	Meaning
Consistent filter (G-2524)	A filter whose real errors are as large as its own reported covariance says, on average; only then can its uncertainty be trusted by a controller or planner.
Covariance matrix (G-495)	A square table with every feature's variance on the diagonal and every pair's covariance off it, so it sums up how the data spreads and which features move together; PCA takes its eigenvectors.
Multivariate normal distribution (G-1283)	The bell-shaped (normal) distribution for several numbers at once, a vector; it is set by a mean vector (its centre) and a covariance matrix (its spread and how the numbers vary together).
Transpose (G-2012)	A matrix or vector with rows and columns swapped; it turns a column vector u into a row u^T, so $u^\mathsf{T}x$ is the dot product.
Inverse matrix (G-968)	The matrix that undoes another: their product is the identity matrix.
Dead reckoning (G-2311)	Working out a robot's position from a known start by adding up its own moves, without looking at its surroundings; simple, but its error grows with every move, so it needs readings to correct it.
Extended Kalman filter (EKF) (G-2520)	The Kalman filter for nonlinear motion and sensor models: it moves the mean through the nonlinear functions and uses their Jacobians at the latest estimate in place of F and H, so it can track unimodal beliefs through nonlinear models.